

Automated Review Rating System

1 Project Overview

An intelligent system that analyzes customer reviews and predicts corresponding star ratings. It uses Natural Language Processing (NLP) to extract sentiment and key features from textual feedback. The model is trained on labeled review data to learn patterns between language and rating. This enables automated, consistent, and scalable review evaluation, helping businesses gain actionable insights and improve decision-making.

2 Environment Setup

- Python 3.9+ was used as the base language.
- Key libraries: Pandas, NumPy, Scikit-learn, Joblib, Matplotlib, Seaborn.
- IDE Used Jupyter Notebook
- Verified proper functioning of packages for data preprocessing, NLP, and data visualization.

3 GitHub Project Setup

The screenshot displays a GitHub repository interface for the user 'Akash-VS2002'. At the top, navigation elements include the repository name, a dropdown for the 'main' branch, a '1 Branch' indicator, a '0 Tags' indicator, a search bar labeled 'Go to file', an 'Add file' button, and a green 'Code' button. Below the repository header, a table lists the files and folders in the repository, including their commit messages and the time since the last commit.

File/Folder	Commit Message	Last Commit
app	Restore deleted app/kaggle folder	last week
data	Add files via upload	now
frontend	Delete frontend/kaggle directory	last week
models	Add folder structure with README and requirements	last week
notebooks	Add folder structure with README and requirements	last week
README.md	Initial project structure with README	last week
requirements.txt	Initial project structure with README	last week

Below the file list, the 'README' file is selected and its content is displayed. The title 'Automated Review Rating System' is prominently shown, followed by a description of the project's purpose and technology stack.

Automated Review Rating System

This project is an Automated Review Rating System that uses Natural Language Processing (NLP) and Machine Learning to analyze customer reviews and generate ratings automatically.

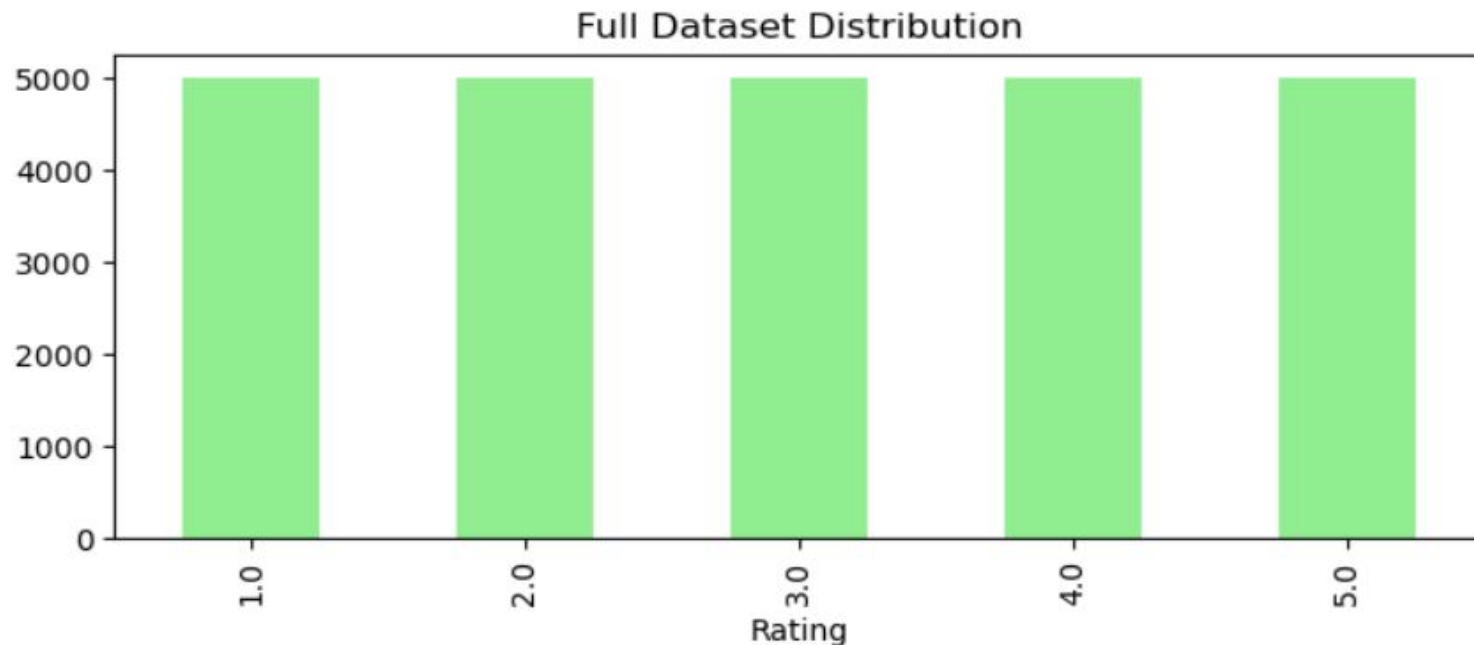
4 Data Collection

Datasets Collected from Kaggle:

- [Amazon Fashion Reviews](#)
- [TripAdvisor Hotel Reviews](#)
- Data includes user reviews and corresponding ratings (1–5 stars)
- Combined datasets to create a unified dataset for analysis
- Ensured balanced representation of ratings for better model training

4.1 Balancing the Dataset

- The dataset contains **25,000 reviews**, balanced with **5,000 samples** for each star rating (1★–5★).
- The **balanced dataset** is stored in 👉 [cleaned_reviews.csv](#)



5 Data Preprocessing

Data preprocessing involves cleaning, transforming, and organizing raw data to make it suitable for analysis or modeling. It ensures **accuracy, consistency, and better model performance** in machine learning tasks.

5.1 Removing Duplicates

Identify and remove duplicate reviews to avoid bias and redundancy in the dataset.

5.2 Detect and Remove Confusing Reviews

Detect and remove reviews where the text does not match the given star rating, ensuring label accuracy.

5.3 Dropping Unnecessary Columns

Remove columns from the dataset that do not contribute to review rating prediction. Reduces noise and improves model training efficiency.

Code 👉 `df = df.drop(columns=['review_id', 'username', 'date'])`

5.4 Handling Missing Values

Identify and handle missing or null values in the dataset to ensure clean and complete data. Missing values in reviews or ratings can negatively affect model training and predictions.

Code 👉 `df = df.dropna()`

5.5 Converting Text to Lowercase

Convert all review text to lowercase to maintain uniformity. Helps the model treat words like **Good** and **good** as the same token, reducing redundancy.

Code 👉 `df['review_text'] = df['review_text'].str.lower()`

5.6 Removing URLs, Emojis, and Special Characters

Clean review text by removing unnecessary elements like URLs, emojis, and special characters.

Helps reduce noise and ensures that the text is suitable for feature extraction and modeling.

Code 👉 `text = re.sub(r'http\S+|www.\S+', '', text), text = text.encode('ascii', 'ignore').decode('ascii'), text = re.sub(r'^a-zA-Z0-9\s', '', text)`

5.7 Removing Punctuations

Remove punctuation marks (like . , ! ? ; :) from review text. Helps reduce noise and ensures that words are treated consistently during feature extraction.

Code 👉 `df['review_text'] = df['review_text'].str.replace(f"[{string.punctuation}]", "", regex=True)`

5.8 Stopword Removal

Stopwords are common words (e.g., “the”, “is”, “and”) that do not contribute significant meaning to text analysis.

Removing them reduces noise and improves model performance for sentiment or review rating prediction. Total stopwords removed: 124557

Removed stopwords

['doing', 'through', 'me', 'those', 'does', 'where', 'some', 'o', 'didn', 'all', 'both', 'or', 'into', 'her', 'did', 'your', 'but', 'who', 'by', 'an', 'should', 'being', 'my', 'other', 'can', 'do', 'won', 'wouldn', 'when', 'after', 're', 'below', 'while', 'no', 'not', 'couldn', 'few', 'we', 'have', 'which', 'as', 'she', 'here', 'then', 'and', 'most', 'more', 'their', 'with', 'up', 'the', 'of', 'once', 'wasn', 'over', 'out', 'its', 'from', 'about', 'nor', 'down', 'each', 'was', 'll', 'are', 'during', 'weren', 'will', 'if', 'above', 'you', 'had', 'these', 'our', 'now', 'y', 'isn', 'myself', 'm', 'there', 'doesn', 'in', 'further', 'theirs', 'it', 'than', 'he', 'aren', 'they', 'to', 'any', 'ain', 'very', 'on', 'has', 'how', 'too', 'having', 'am', 'ourselves', 'were', 'at', 'between', 'this', 'own', 'what', 'them', 'haven', 'same', 't', 'hasn', 'until', 'that', 'be', 'been', 's', 'for', 'why', 'off', 'a', 'don', 'under', 'shouldn', 'd', 've', 'before', 'is', 'i', 'again', 'so', 'only', 'because', 'itself', 'ma', 'just', 'hadn', 'such']

Code 🙌

```
def remove_stopwords(text):  
    text = str(text).lower()  
    text = re.sub(r'^a-z\s]', "", text)  
    words = text.split()  
    cleaned_words = [lemmatizer.lemmatize(w) for w in words if w not in stop_words]  
    return " ".join(cleaned_words)  
  
df = pd.read_csv(r"C:/Users/MY PC/OneDrive/Desktop/kaggle/balanced_reviews.csv")  
df['Cleaned_Review'] = df['Review'].apply(remove_stopwords)
```

5.9 Lemmatization

Lemmatization is the process of reducing words to their dictionary or base form (lemma). It considers the context and meaning of the word. Unlike simple stemming, lemmatization produces real words that exist in the language. This helps in preserving the semantic meaning of the text, which improves the accuracy of NLP tasks like sentiment analysis, text classification, and review rating prediction.

Example:

- Words like **running**, **ran**, **runs** → lemmatized to **run**
- Words like **better** → lemmatized to **good**

Why Lemmatization is Better than Stemming

meaning Preservation:

- Lemmatization reduces words to their **real dictionary form (lemma)**, preserving the meaning.
- Stemming simply cuts word endings, which can produce non-words or incorrect forms.

Accuracy:

- Lemmatization considers the **context and part of speech**, making it more accurate.
- Stemming uses a heuristic approach, often leading to errors.

Example:

- Lemmatization: **running** → **run**, **better** → **good**
- Stemming: **running** → **run**, **better** → **better** (unchanged, may be incorrect)

5.10 Filter by Word Count

Filtering reviews by word count ensures that extremely short or empty reviews, which may not provide meaningful information, are removed. This improves model training by focusing on reviews with enough content to learn patterns.

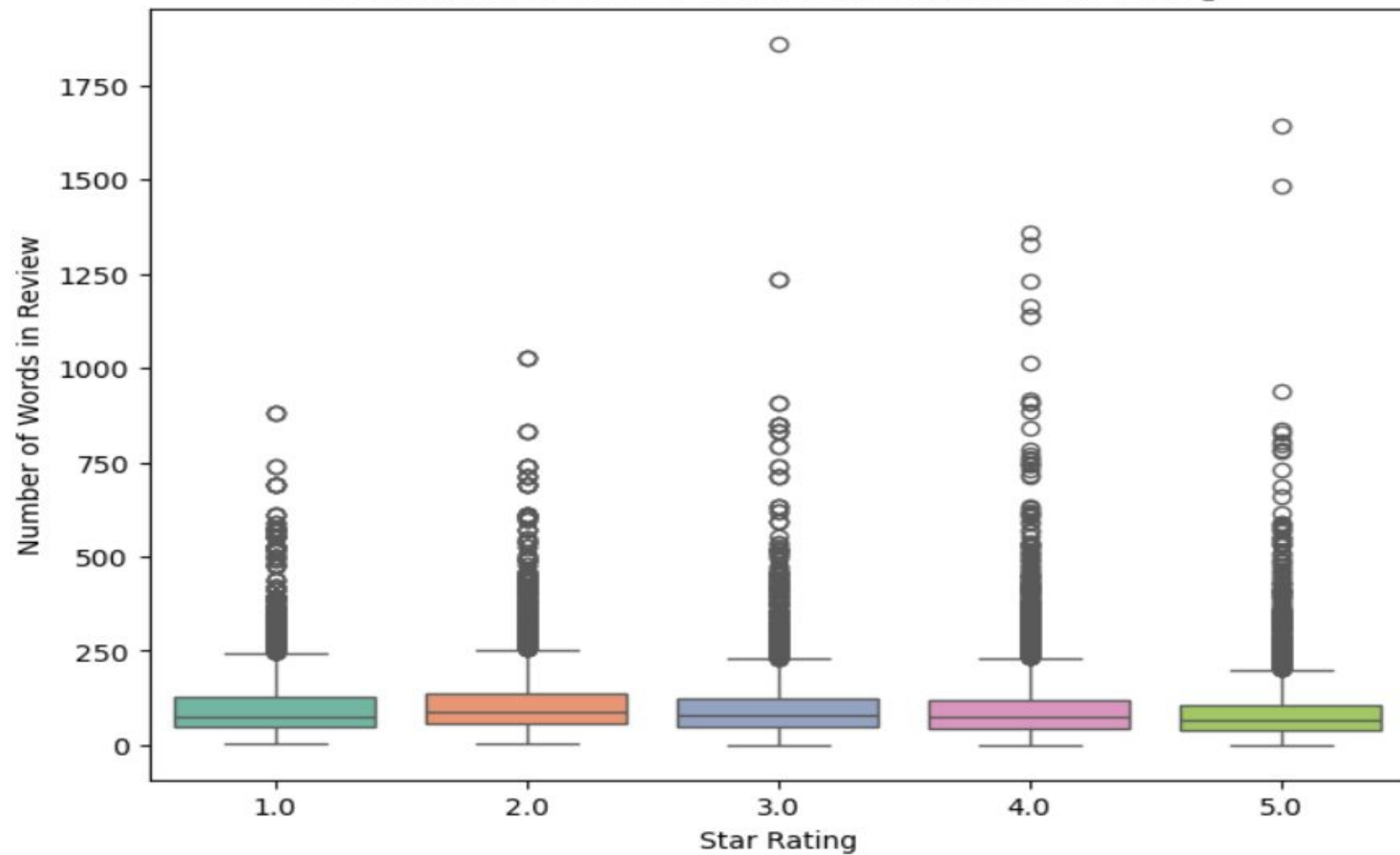
6 Data Visualization

Data visualization helps to **understand the dataset**, identify patterns, and communicate insights effectively before model training. In the Automated Review Rating System, we visualize reviews, ratings, and text characteristics.

Box Plot

A **box plot** is a visual tool used to display the distribution of numerical data, showing the **median, quartiles, and potential outliers**. It helps quickly understand how data is spread and identify any extreme values. In this project, **box plots were used to compare the word count distribution of reviews across different star rating classes**, allowing us to analyze patterns such as whether longer reviews tend to correspond to higher or lower ratings.

Distribution of Review Word Count Across Star Ratings



Sample of 1-Star Rating

1. bad experience stay stayed week waikiki marriott horrible experience noise terrible room really run option waikiki stay
2. worst hotel room awful window poor lighting extremely cramped poor bedding door bathroom screen bathroom extremely basic stay hotel circumstance
3. less week sole ripped outside shoe
4. lesson learned air conditioner dont degree mildew smell seedynasty staff low class clientle scene problem evidently used time stay marriott
5. filthy smell hotel building gutted torn downit significant plumbingsewage problem produce strong odor east wing buildingexteriorstreet room noisy

Sample of 2-Star Rating

1. city pick pocket basic hotel room cleaned daily facility reception staff came shot dawn personallities got robbed day told acted nt toss
2. day inn stayed past year ragedy bed bathroom clean wore hotel usually noisey single pane window rowdy group staying hotel good price settle noisey clean hotel
3. hotel pro great location waikiki beach reasonable pricecons oldoutdated hotel small pool concierge outsourced knowledgeable area
4. ok staff friendly staff friendly poop bathroom checked room great location bus stop right outside
5. shoe poorly constructed dont expect last one summer would recommend order another nike downshifter series

Sample of 3-Star Rating

1. work professional helpful reservation hotel problem place fit need desk credit professional understanding circumstance forced cancel reservation
2. average airplane noise hotel average certainly worth usd night tiny beach pool day long hear single airplane taking landing hotel mile away room clean dvd player nice touch stay
3. waaay small use futur child
4. hotel room small great view parking ramp block away vehicle extra park ramp addition regular hotel booking san francisco really sure area really nice hotel employee helpful telling town
5. gone downhill bit hotel nice declined stay obvious example plasticwrapped plastic cup bathroom compared glass service good convenient location good choice downtown

Sample of 4-Star Rating

1. great experience special thanks darlene beautiful room fireplace comforting nestled bed watch flickering flame fireplacei recommend queen anne sure return
2. friendly staff family stayed springhill suite husband attended conference chose hotel based recommendation review nt disappointed easy walk nice room friendly staff definitely stay
3. great hotel stayed estherea oct chose hotel great review disappointed decent sized room appointed comfy bed large bathroom amsterdam time estherea far nicest hotel stayed best location definitely stay
4. walk stayed day october motel location close train station hotel walk local sight hotel elevator moving luggage trying breakfast good staff helpful noise street ending scooter enjoyed minute
5. nice breakfast assortment clean room important standard best continental breakfast encountered plenty high quality frequent maintenance early crowd

Sample of 5-Star Rating

1. pleasantly surprised great hotel booked internet cheap special nt expecting greatest room wonderful better thought say best room stayed inclean quite room lovely bathroom huge spotlessly clean little extra darling harbour perfect road chinatown perfect eating friendswould love stay hope future
2. wonderful place celebrate th anniversary place best really quiet staff perfect loved ability walk snack day wanted wine cheese social eveningi wanted make th anniversary special casablanca happen againjohn kori
3. comfortable look great
4. fantastic returned night city break friend absolutely loved position hotel central hotel spotlessly clean breakfast wonderful terrace overlooking la duoma morning definetely recommend
5. nice place tme mexico city amazed progressive hotel business conference ideal tourist thing lack pool

7 Train–Test Split

The dataset is divided into two parts:

- **Training set (80%)** → used to train the model.
- **Testing set (20%)** → used to evaluate model performance on unseen data.

Stratified sampling is applied to ensure that all star rating classes (1 to 5) are proportionally represented in both training and testing sets.

This prevents the model from becoming biased towards any specific rating class.

How It Was Done

The dataset contains 25,000 reviews, evenly distributed across five rating classes, with 5,000 reviews per class. For effective model training and evaluation, the dataset was split into 20,000 training samples (80%) and 5,000 testing samples (20%). The `train_test_split` function was applied with the `stratify` parameter, ensuring class balance. This preserved equal representation, with each rating contributing 4,000 samples to training and 1,000 to testing. Such stratified sampling helps prevent class imbalance, ensuring fair evaluation across all rating categories.

Training Dataset: [Training Dataset](#)

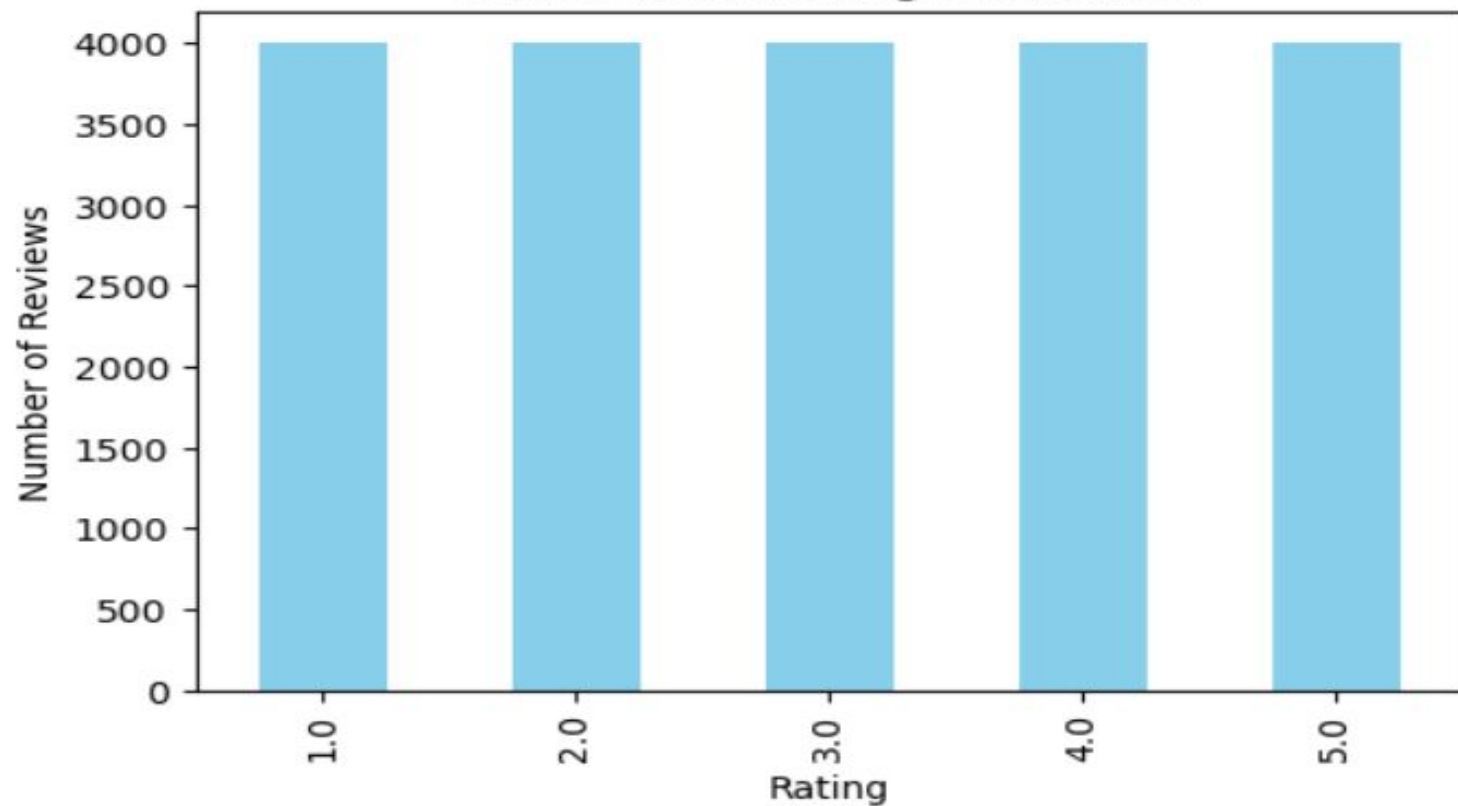
Testing Dataset: [Testing Dataset](#)

Code 🙌 `X = df['Cleaned_Review'] # or df['Review'] if you want raw text`
`y = df['Rating']`

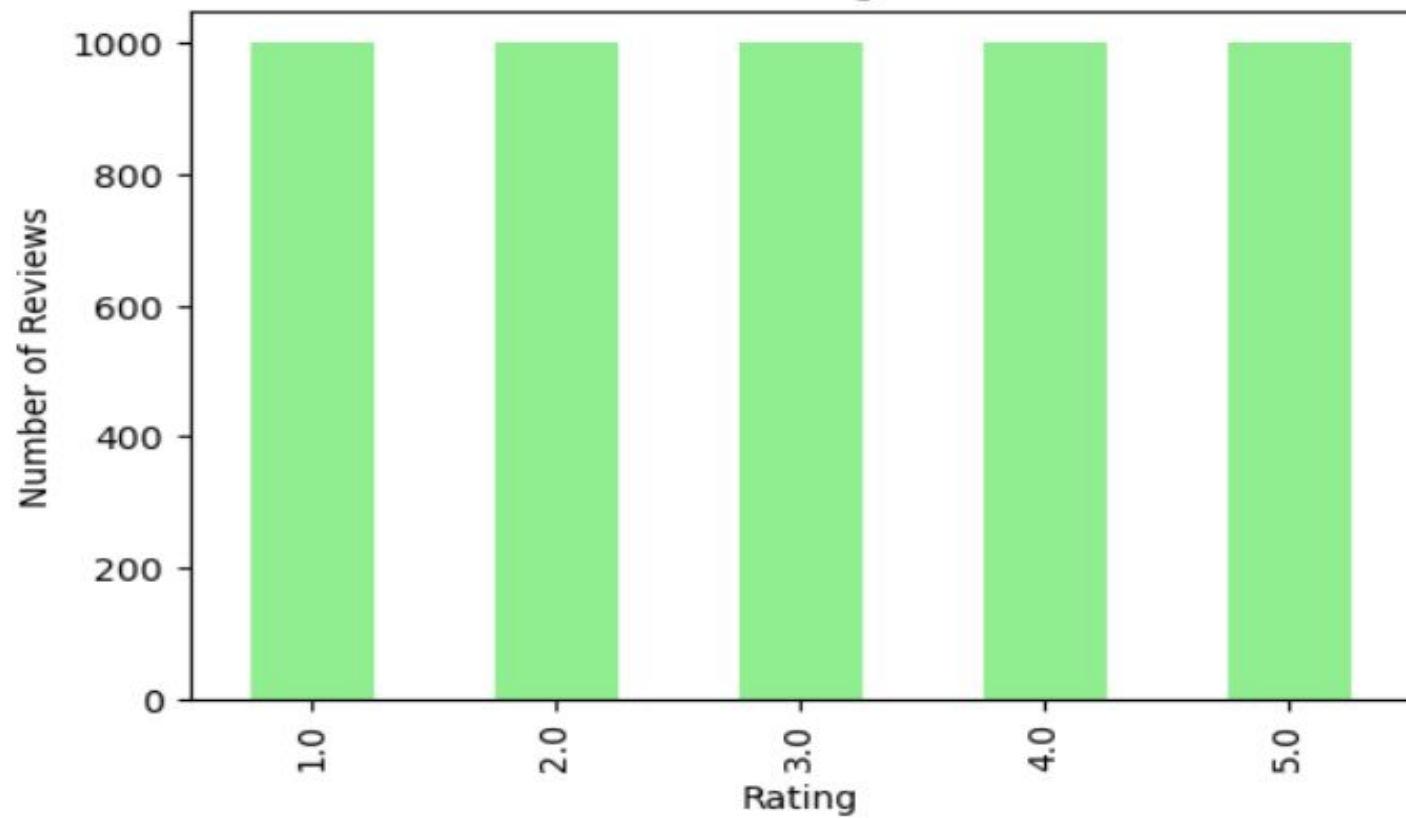
`# Train-test split (80% train, 20% test) with stratification`

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y,  
    test_size=0.2,  
    stratify=y,  
    random_state=42  
)
```

Train Dataset Rating Distribution



Test Dataset Rating Distribution



8 Text Vectorization

Text vectorization transforms textual reviews into numerical representations that machine learning models can process. It converts words into features, capturing patterns and relationships within the text for effective model training.

TF-IDF (Term Frequency–Inverse Document Frequency)

TF-IDF is a text vectorization technique that converts words into numerical values by combining **Term Frequency (TF)** and **Inverse Document Frequency (IDF)**. It highlights words that are important in a document while down-weighting common, less informative words.

Formula For TF-IDF

$$TF(t, d) = \frac{\text{number of times } t \text{ appears in } d}{\text{total number of terms in } d}$$

$$IDF(t) = \log \frac{N}{1 + df}$$

$$TF - IDF(t, d) = TF(t, d) * IDF(t)$$

Code 🙌 `X_train = tfidf.fit_transform(train_df['Review'])`

`X_test = tfidf.transform(test_df['Review'])`

`y_train = train_df['Rating']`

`y_test = test_df['Rating']`

Output 🙌

	aaa	abc	ability	able	abroad	absolute	absolutely	absolutley	absolutly	abundance	...	younger	yr	yuck	yuk	yummy	yunque	zero	zona	zone	zoo
0	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	0.0	0.0	0.0	0.073896	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
19995	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
19996	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
19997	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
19998	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
19999	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

20000 rows × 5000 columns

9 Model Training

Objective: Train multiple machine learning models on vectorized text data to predict user ratings. Evaluate and select the best-performing model.

Models Trained

- **Logistic Regression**
- **Linear SVM (LinearSVC)**
- **Multinomial Naive Bayes**
- **Decision Tree Classifier**
- **Linear Regression**

```
Code 👉 models = {  
    "Logistic Regression": LogisticRegression(max_iter=1000),  
    "Linear SVM": LinearSVC(),  
    "Multinomial NB": MultinomialNB(),  
    "Decision Tree": DecisionTreeClassifier(),  
    "Random Forest": RandomForestClassifier()  
}
```

```
for name, model in models.items():  
    model.fit(X_train, y_train)  
    y_pred = model.predict(X_test)  
    print(f"--- {name} ---")  
    print("Accuracy:", accuracy_score(y_test, y_pred))  
    print(classification_report(y_test, y_pred))
```

Model Name	Accuracy	Precision (Macro Avg)	Recall (Macro Avg)	F1-Score (Macro Avg)
Logistic Regression	0.4866	0.48	0.49	0.48
Linear Regression	0.2182	nan	nan	nan
SVM	0.418	0.42	0.42	0.42
Naive Bayes	0.4724	0.46	0.47	0.47
Decision Tree	0.3258	0.32	0.33	0.32

9.1 Model Fine-Tuning

“Model fine-tuning optimizes hyperparameters to improve predictive performance, reduce overfitting, and enhance generalization. GridSearchCV systematically tests parameter combinations, selecting the best model configuration for higher accuracy and F1-score.”

Code 🙌

```
param_grid_lr = {  
    'C': [0.1, 1, 10],  
    'penalty': ['l2'],  
    'solver': ['lbfgs', 'saga']  
}  
grid_lr = GridSearchCV(LogisticRegression(max_iter=1000), param_grid_lr, cv=5, scoring='f1_macro')  
grid_lr.fit(X_train, y_train)  
best_lr = grid_lr.best_estimator_
```

Output 🙌  Final Comparison (Balanced Dataset):

Logistic Regression | Accuracy: 0.4926 | Params: C=1

Linear SVM | Accuracy: 0.4918 | Params: C=0.1

MultinomialNB | Accuracy: 0.4724 | Params: alpha=1.0

🏆 Best Model: Logistic Regression → Accuracy: 0.4926 (C=1)

9.2 Why Logistic Regression Was Chosen

Highest Accuracy:

Logistic Regression achieved **0.4866**, higher than all other models, making it the most reliable overall predictor.

Balanced Class Performance:

- Macro F1-score **0.48** is the highest among tested models.
- It predicts both minority and majority classes better than Naive Bayes or Random Forest.

Better Generalization:

- Less overfitting compared to Decision Tree and Random Forest.
- Handles sparse TF-IDF features efficiently, unlike SVM which can be sensitive to parameters.
- **Stable and Interpretable:**
- Coefficients provide insights on which words affect ratings.
- Fast training makes it suitable for iterative fine-tuning.

9.3 Final Model Prediction

S.No	Actual Rating	Predicted Rating	Review (First 200 chars)
1	1	3	This product tastes good, but you can buy a container of each type for \$9 at Wegman's. If you get it from this place, you pay \$26 (TWENTY SIX!) for shipping and handling. Why? All I found was a freeze...
2	1	1	Several very dented cans, even in the middle of the package. I had to throw out 4 cans out of 12 because they were so dented I didn't trust that the soup inside was safe to eat.
3	1	1	I ORDERED THIS CHOCOLATE ON FEBUARY 18,2012. IT WAS 28.99. IT WENT UP 10 BUCKS AND ALMOST 7 BUCKS TO SHIP. THE PRICE OF COCOA DIDNOT RISE THAT MUCH. SOMEONE MUST HAVE GOTTEN A LARGE RAISE. PLEASE BRIN...
4	1	1	Go with beef bones or marrow bones. The salt they put in the pork bones and other chemicals to keep them from rotting is horrible for your dogs. Smell one next time you go to feed one to your dog, i...
5	1	1	You won't find yourself soaking up the last trace of Scarpetta pasta sauce on your plate with a slice of pugliese. In fact, I would be surprised if you even made it through the meal. I'll h...
6	2	2	Japanese green tea has a cleaner, grassier flavor than the Chinese variety. You don't get as much tea for the money with this one as the less expensive brands, and I find the flavor to be too subtle....
7	2	1	I discovered Goya's Pink Beans at a local liquidation store. Tried them and loved them! I decided to check Amazon to see if they had them. Amazon had them listed as 4 pound bags so I snat...
8	2	2	I only bought one package of these, since they were on sale for very cheap. We go through 2 fruit and veggie pouches every day, so I'm always on the lookout for a deal. A few of the pouche...
9	2	3	The fish is ok but the sodium level is very high so you taste mostly salt. My suggestion is to order a sockeye product with no salt added such as Golden Crown and then you can always add seasoning to ...
10	2	1	Okay...this ended up being a \$12 bag of salty, baked cheese. Seriously?! \$6 for the bag and then \$6 to ship it. I didn't even realize until it showed up and I saw the packing slip!! Certa...
11	3	1	These are very good but it does still contain sugar alcohols and it can create bad gas and or worse.
12	3	3	This conditioner is very thick and rich. It smells really good and rinses easily. The moisturizing and detangling ability is fair at best, though. I would have liked to see better performance in those...
13	3	2	Having reviewed the other two Taco Bell sauces, I felt I had to complete the trifecta, and so ended up with this one. It was everything I expected: kind-of orange ranch dressing with a little bit of ...
14	3	3	The beef pot roast with potatoes and carrots is pretty middle of the line for the Hormel meals. The thing that makes me like the red meat meals less than some of the chicken and pasta meals is that th...
15	3	1	Not bad cake but happily my homemade is still better.
16	4	4	As good as the regular steeping-bag tea of the same name. A little higher cost but convenience of K-Cup is hard to beat. I would like to find larger quantity at lower unit cost.
17	4	4	The ingredients seem good quality, although nothing is recognizable in the can, like with Merrick. It's great for mixing with kibble to moisten it and if your dog needs mush--soft wet, canned food.
18	4	4	best beach stayed bavaro princess suites march 4 11 2006 apple vacations, hotel beautiful grounds especially clean service good food used dominican republic, no complaints, beach following advantages ...
19	4	3	if i didnt need to be gluten free, i would have never tried this product. who knew corn pasta would be any good? but i liked the corn spaghetti i had gotten (no idea the brand) so i decided to try thi...
20	4	5	Of course my fat dog loved these treats! It's always good to feed your pooch something that might be good for her, rather than something that might cause allergies or other issues. I'm hoping the "org...
21	5	5	I eat this everyday for breakfast - I soak the oats in water overnight, then it only takes 1/2 the time to cook in the morning. I add raisins, flax meal, bite size pieces of granny smith apple, and ci...
22	5	4	Trident White makes a terrific "cool rush" flavored sugarless gum. The sugar free gum is healthier for your teeth; and the gum makes your breath fresh in an instant. The thin cardboard package is easy...
23	5	4	This soup is just good. The broth is spot on and the corn with the other vegetables complement one another very well. I have to confess that I have a bad habit of putting a little pat of butter in t...
24	5	5	great hotel good position stayed 2 nights mainly universal arrived pleasantly supried fantastic facilities literally complex.the hotel rooms beautifully decorated clean, hollywood highland complex gr...
25	5	4	My first venture into purchasing freeze-dried Wakame, I have been very pleased with this product. I bought it primarily for making Miso soup. It is certainly good enough for other uses as well. Hydrat...

10 Model Testing on Cross Test Sets

Evaluate pre-trained models on **cross test sets** to understand how **training data distribution (balanced vs imbalanced)** affects performance.

10.1 Load Model & Test Data

Code 👉 `Model_A = joblib.load('Model_A.pkl')`

`test_imb = pd.read_csv('test.csv') # Imbalanced test`

`X_test_imb = test_Imb['Review'].fillna("no review").astype(str)`

`y_test_imb= test_Imb['Rating']`

Result

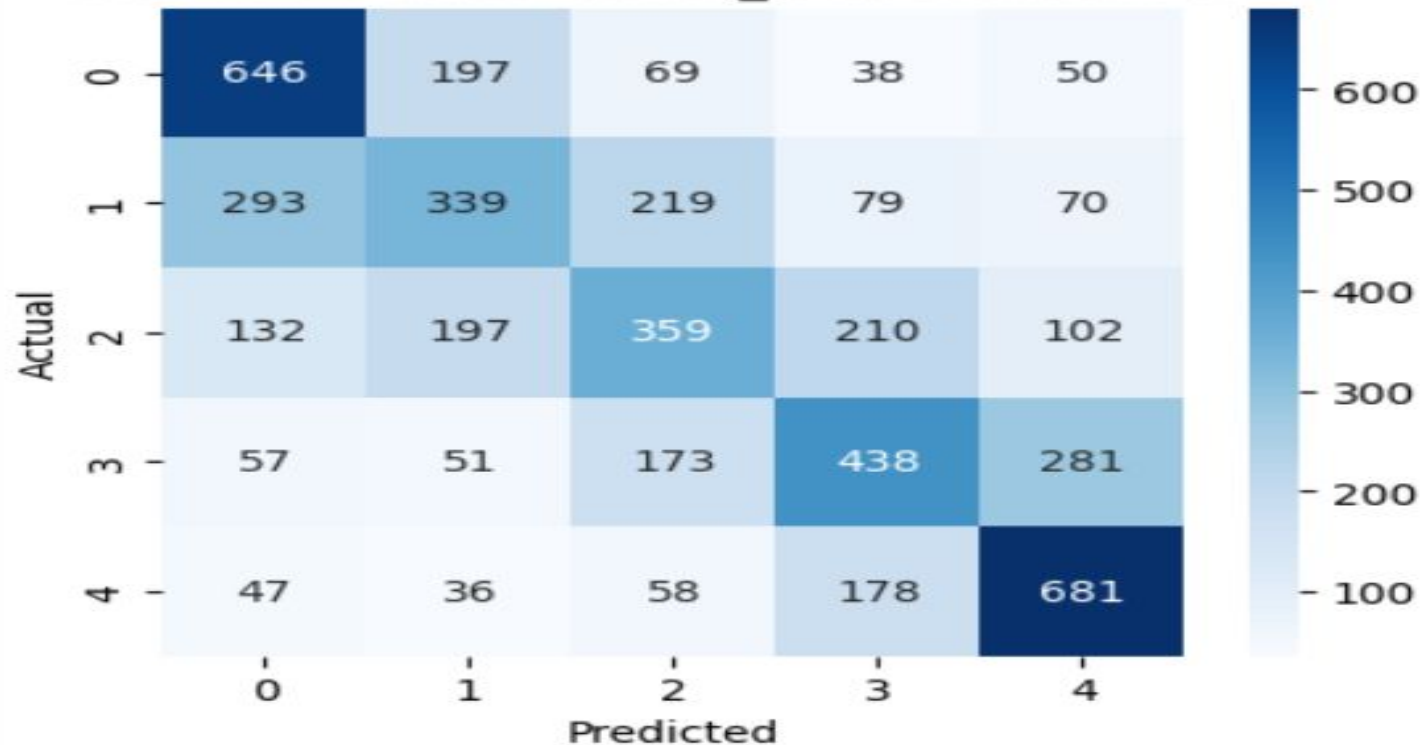
=== Model_A on Balanced Test ===

Accuracy: 0.4926

Classification Report:

	precision	recall	f1-score	support
1	0.55	0.65	0.59	1000
2	0.41	0.34	0.37	1000
3	0.41	0.36	0.38	1000
4	0.46	0.44	0.45	1000
5	0.58	0.68	0.62	1000
accuracy			0.49	5000
macro avg	0.48	0.49	0.48	5000
weighted avg	0.48	0.49	0.48	5000

Confusion Matrix - Model_A on Balanced Test



=== Model_A on Imbalanced Test ===

Accuracy: 0.4082

Classification Report:

	precision	recall	f1-score	support
1	0.31	0.64	0.41	500
2	0.29	0.23	0.26	750
3	0.43	0.30	0.35	1250
4	0.48	0.36	0.41	1500
5	0.47	0.63	0.54	1000
accuracy			0.41	5000
macro avg	0.39	0.43	0.39	5000
weighted avg	0.42	0.41	0.40	5000

Confusion Matrix - Model_A on Imbalanced Test

